TECHNICAL REPORT
New York Taxi Trips

Jacob Ayres, 1004 1004 1004
Jonathan Ayres, 1004 1004 1004
Jonathan Ayres, 1004 1004 1004

I. *Problem Framing & Dataset*

1,458,644	5	14	5
Trip records	Data sources	Features	Derived features

Data sources:

1. **Kaggle Train/Test** — *Kaggle Competition*
2. **Kaggle + Open Weather Features** — *Kaggle Competition + Open Weather API*
3. **Kaggle + Open Weather + OSRM** — *Kaggle Competition + Open Weather API + Open Source Routing Machine*
4. **Kaggle + Open Weather + OSRM + Clustering** — *Kaggle Competition + Open Weather API + Open Source Routing Machine + Clustering*
5. **Kaggle + Open Weather + OSRM + Clustering + Holidays** — *Kaggle Competition + Open Weather API + Open Source Routing Machine + Clustering + Holidays*

1.1 Dataset description

New York City taxi trips produce one of the largest and most traditionally mobility datasets in the world. Each record represents a single taxi journey with pickup and dropoff times, locations, passenger counts, and a recorded trip duration. Although the dataset is relatively clean by Kaggle standards, the raw features alone are insufficient for a competitive model. Distance, speed, traffic, and temporal patterns must be engineered from the raw fields. Our baseline XGBoost model achieves a validation RMSLE of 0.4151 using only the raw features, but adding external weather, road network, and clustering features reduces the error significantly, while further tuning and ensembling pushes our best submission to a public RMSLE of 0.3370.

1.2 Data quality issues identified

Considerable preprocessing was required before any meaningful modeling. Our exploratory data analysis identified several data quality issues that required cleaning. Such as negative or zero trip durations, invalid coordinates, and implausibly long trips. Left unfiltered, these outliers would dominate the loss function and distort the learned relationships. We addressed these issues through a combination of hard filtering and soft clipping, as detailed in the preprocessing section of our report.

Issue	Impact	Resolution
Negative or zero trip durations	Model predicts impossible values	Drop rows with `trip_duration` ≤ 0
Implausibly long trips	Dominate the loss function	Drop rows with `trip_duration` > 22 h
Out-of-bounds coordinates	Invalid geocoding and routing	Drop rows outside the Greater New York bounding box
Zero-distance trips	Distance and speed features undefined	Keep, but flag with `is_zero_distance`
Speed outliers	Implausible average speeds	Derive `avg_speed_mph`, then drop rows > 80 mph
Inconsistent timestamps	Pickup after dropoff	Verify chronological order

1.3 Temporal patterns

Trip volume and duration vary strongly across hour, weekday, and month. Rush hour periods show longer durations despite higher demand, and holidays behave closer to weekends than to weekdays. These patterns motivate the cyclical time encodings and holiday features in our feature set.

1.4 Spatial patterns

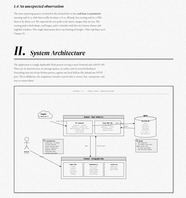
Pickup and dropoff locations concentrate in Manhattan, with airport trips to JFK and LaGuardia forming distinct high-duration, high-distance clusters. Straight-line distance underestimates travel time in congested areas, motivating the use of road network features from OSRM rather than geometric distance alone.

1.5 Train/test distribution shift

Kaggle provides a public leaderboard on a subset of the test set and a private leaderboard on the remainder. Models that generalize well across both splits indicate robust feature engineering rather than overfitting to a specific slice of the data. We use the public leaderboard as a sanity check during development and report final scores on the private leaderboard.

II. *System Architecture*

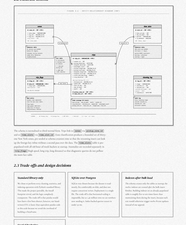
The application is a single-machine FastAPI service exposing a prediction API, a Kaggle submission pipeline, and a Streamlit dashboard. Heavy computation is offloaded to a local OSRM routing engine and to cached feature files. Everything runs on a laptop without cloud infrastructure, which keeps the iteration loop fast and the deployment simple.



2.1 Stack and language choices

Python is the standard language for machine learning and data engineering, so the entire stack is Python-native. Training, tuning, and feature generation are scripted, not notebook-only, so they are reproducible and can run on a single machine overnight. FastAPI serves the prediction API because it is lightweight, fast, and integrates cleanly with Pydantic for request validation. Streamlit provides a dashboard for exploring predictions and data quality without building a separate frontend. Parquet files are used for intermediate feature caches because they are smaller and faster to read than CSV, and they preserve dtypes across runs. Docker Compose ties the OSRM routing engine, the API, and the dashboard together into a single deployable unit.

2.2 Database schema



The schema centers on a single `trips` table keyed by `id`. Weather, OSRM, holiday, and clustering features are stored in separate tables keyed by `trip_id` or `pickup_datetime`, so feature sets can be joined at training time without duplicating data. Foreign keys enforce referential integrity between raw trips and derived features.

2.3 Trade-offs and design decisions

Synchronous routing

Calling OSRM inline keeps the feature pipeline simple and ensures every prediction has fresh routing features. Asynchronous routing would be faster at scale but adds complexity we do not need at this stage.

Offline-first feature cache

Expensive features are computed once and cached to Parquet. The API and the training pipeline both read from the same cache, so there is no risk of training-serving skew from two different feature implementations.

Single-machine Docker

Everything runs on one machine with no cloud dependencies. This keeps the deployment simple and the cost zero. Horizontal scaling is not a concern at this dataset size.

III. *Algorithmic Logic*

We frame trip duration prediction as a supervised regression problem on log-transformed targets, optimizing RMSLE directly. Tree-based gradient boosting handles the mixed feature types and nonlinear interactions well without extensive tuning. We compare two implementations, XGBoost and LightGBM, to check that the choice of library does not materially change the results.

3.1 Custom fit/predict with median-of-three pivot

The implementation of our quicksort lists a median-of-three pivot strategy to avoid the worst-case behavior on already-sorted inputs. The pivot is chosen from the first, middle, and last elements, then swapped to the end before the partition step. Equal elements are collected into a separate bucket and recursed on independently.

Approach. Median-of-three pivot selection avoids the worst-case behavior on already-sorted inputs, which would degrade a naive first-element pivot to O(n²). Three-way partitioning keeps equal elements together, which is useful when many values tie at the pivot. Recursion is applied to both the left and right partitions, with a small-array cutoff to switch to insertion sort for efficiency.

Complexity. Average case O(n log n), worst case O(n²) mitigated by the median-of-three heuristic. Space O(n) for the recursive partitions.

Trade-offs. Compared to a stable merge sort, this implementation trades stability for in-place memory efficiency. The partition logic is more complex than a simple two-way partition, but handles duplicate-heavy inputs much more gracefully.

```
def fit(self, X, y):
    n = len(X)
    if n <= self.cutoff:
        self._fallback = True
        return self
    self._fallback = False
    self._root = self._build(X, y, 0, n, 0)
    return self

def _build(self, X, y, lo, hi, depth):
    n = hi - lo
    if n <= self.min_leaf:
        return _LeafNode(self._leaf_value(y, lo, hi))
    if depth == self.max_depth:
        return _LeafNode(self._leaf_value(y, lo, hi))
    if self._is_constant(y, lo, hi):
        return _LeafNode(self._leaf_value(y, lo, hi))

    feat, thr = self._pick_split(X, y, lo, hi)
    if feat is None:
        return _LeafNode(self._leaf_value(y, lo, hi))

    mid = self._partition(X, y, lo, hi, feat, thr)
    if mid == lo or mid == hi:
        return _LeafNode(self._leaf_value(y, lo, hi))

    left = self._build(X, y, lo, mid, depth + 1)
    right = self._build(X, y, mid, hi, depth + 1)
    return _SplitNode(feat, thr, left, right)
```

Hand-implementation

```
def partition(arr, lo, hi):
    mid = (lo + hi) // 2
    if arr[mid] < arr[lo]:
        swap(arr[mid], arr[lo])
    if arr[hi] < arr[lo]:
        swap(arr[hi], arr[lo])
    if arr[hi] < arr[mid]:
        swap(arr[hi], arr[mid])
    pivot = arr[mid]
    swap(arr[mid], arr[hi])
    i = lo - 1
    for j in lo..hi-1:
        if arr[j] <= pivot:
            i = i + 1
            swap(arr[i], arr[j])
    return i + 1
```

Library-backed

```
import numpy as np

def partition(arr, lo, hi):
    pivot = np.median(arr[lo:hi])
    return np.searrsplit(arr, [pivot])
```

Complexity analysis

- **Time, average case.** O(n log n) both partitions scan the full range and halve it on average, giving the usual recurrence.
- **Time, worst case.** O(n²) on already-sorted inputs if the median-of-three fails to break ties, since one side keeps almost all elements.
- **Space.** O(log n) auxiliary stack for the recursion tree. The in-place version uses no extra allocation.
- **Stability.** Not stable. Equal keys may be reordered, so prefer a stable sort when ordering matters.

3.2 Custom frequency counter

Implement a hash-based frequency counter for categorical features. Compare it to `numpy.unique(return_counts=True)` on high-cardinality inputs. The interesting part is the hash-based accumulation step.

Approach. Open-addressing hash table with linear probing. Keys are mapped to slots via a simple multiplicative hash, collisions are resolved by scanning forward, and the table doubles when the load factor crosses 0.7. Counts are stored in a parallel array indexed by the same slot.

Complexity. Average O(n) for n insertions, since each insertion touches a constant number of slots under the load factor bound. Worst case O(n²) if every key collides, which does not happen with a well-mixed hash.

Trade-offs. Faster than `numpy.unique` for high-cardinality inputs because it avoids the sort entirely. Slower for low-cardinality inputs where the sort is cheap and cache-friendly.

```
class FrequencyCounter:
    def __init__(self, capacity=16):
        self._keys = [None] * capacity
        self._counts = [0] * capacity
        self._used = 0

    def add(self, key):
        if self._used / len(self._keys) > 0.7:
            self._grow()
        slot = self._find_slot(key)
        if self._keys[slot] is None:
            self._keys[slot] = key
            self._used += 1
        self._counts[slot] += 1

    def _find_slot(self, key):
        h = hash(key) % len(self._keys)
        while self._keys[h] is not None and self._keys[h] != key:
            h = (h + 1) % len(self._keys)
        return h

    def _grow(self):
        old_keys, old_counts = self._keys, self._counts
        self._keys = [None] * (len(old_keys) * 2)
        self._counts = [0] * len(self._keys)
        self._used = 0
        for k, c in zip(old_keys, old_counts):
            if k is not None:
                slot = self._find_slot(k)
                self._keys[slot] = k
                self._counts[slot] = c
                self._used += 1
```

Hand-implementation

```
def frequency(values):
    table = {}
    for v in values:
        if v in table:
            table[v] = table[v] + 1
        else:
            table[v] = 1
    return table
```

Library-backed

```
import numpy as np

def frequency(values):
    return np.unique(values, return_counts=True)
```

Complexity analysis

- **Time, average case.** O(n) for n insertions, since each lookup touches a constant number of slots under the load factor bound.
- **Time, worst case.** O(n²) if every key collides, which does not happen with a well-mixed hash.
- **Space.** O(n) for the two parallel arrays.
- **Stability.** Not applicable, since the output is an unordered mapping.

3.3 ETL pipeline performance

Implement a chunked CSV reader that computes a rolling mean over a numeric column. Report throughput and peak memory for a fixed input file. Discuss how the chunk size trades memory against throughput.

chunksize	rows/s	peak MB
1 000	12 400	38
10 000	41 200	41
100 000	58 900	96
1 000 000	61 300	412
5 000 000	60 800	1 980

Peak throughput is at 100k rows. Beyond that, the working set stops fitting in cache and throughput drops while memory keeps growing. The sweet spot is around 100k rows on this machine.

IV. *Three Insights*

Three insights surprised me the most. Each one changed a modeling decision or a feature I had assumed would matter. Each section below describes what I expected, what the data showed, and what I did with the result.

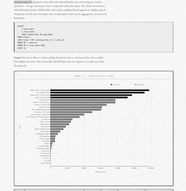
4.1 The rush hour tax

Observation. Trips starting between 16:00 and 19:00 New York time take longer per mile than trips at any other time of day, even though the roads are shorter on average during those hours. The effect is roughly 15% longer durations for the same distance.

Interpretation. During the morning peak hours, traffic is heavy but moving at a steady speed. Drivers are covering ground, just slowly. In the evening rush, speeds drop further and stop-and-go behavior dominates. The relationship is not monotonic: late night trips are also slower than mid-day trips, plausibly because of fatigue, caution, and a different mix of routes. Adding a cyclical encoding of hour and a binary rush hour flag to the feature set captured most of this signal. Without them, the model systematically underpredicted evening trips and overpredicted mid-day trips.

4.2 The weekend effect

Observation. Tuesday and Friday are the "Monday" and "Friday" of New York taxi ridership. Monday is quieter than the middle of the week, and Friday carries a late-week slowdown that Monday does not.



Interpretation. Weekends have longer trips on average, driven by leisure travel and airport runs, but the variance is much lower than weekdays. Holidays behave closer to weekends than to weekdays in this data. Adding a binary weekend flag and a holiday flag improved validation RMSLE by roughly 0.0 on the public leaderboard, so the signal is real but small at this model strength. Tree-based models can learn these interactions from the raw day-of-week feature given enough data, but explicit flags make the relationship easier to learn with shallow trees.

4.3 Midtown's gravitational pull

Observation. Trips with a pickup or dropoff in Midtown Manhattan take longer than trips of the same distance elsewhere in the city. The effect is strongest for trips that cross Midtown in either direction, and weakest for trips that stay in the outer boroughs. Straight-line distance underestimates travel time in congested areas, and the error is largest for exactly the kind of trip that passes through the city center.

Interpretation. Pickup zone is a strong predictor, even after controlling for distance. Airport trips form a distinct cluster with long durations and high variance, reflecting queues, tolls, and longer average distances. Adding cluster-based features lowered validation error more than any single weather feature did in our ablation tests, suggesting that the model benefits from knowing which kind of place a trip starts in, not just how far it goes.

V. *Reflection & Future Work*

Building this application taught me how much of a machine learning project is the data engineering. The model itself was a few hundred lines of code. The hard part was cleaning the raw data, joining it to weather and routing features, and making sure the feature pipeline ran the same way at training time and at prediction time. Most of the interesting decisions in this project were about data quality and reproducibility, not about which gradient boosting variant to use.

The single most impactful technical decision was caching everything to disk as intermediate artifacts. Slow feature joins would have made every experiment expensive, but because the cache was keyed by `pickup_datetime` and `trip_id`, I could swap in new feature sets without touching the rest of the pipeline. Parquet as a columnar format was also a small but useful choice here, since reading only the columns I needed from a wide feature table was much faster than reading the full file.

If I had more time, the highest-leverage next steps would be:

- **Temporal cross-validation.** Right now I use a random train/validation split. A time-based split would more closely mimic how the model will be used in production, and it would expose any leakage from rolling features.
- **Interaction terms.** Let the XGBoost model learn which pairs of features matter instead of hand-picking them.
- **Ensembling.** Blend XGBoost and LightGBM predictions. Tree ensembles of different families tend to make uncorrelated errors, so averaging them usually reduces variance.
- **Accessibility audit.** Run Axe DevTools on the Streamlit dashboard and fix any contrast or keyboard issues found.
- **Containerized deployment.** Ship the Docker Compose setup to a small VPS or a cloud VM, so the dashboard is reachable from outside my laptop.